Genomics

Leukemia is a cancer of blood-generating tissues. Over 475,000 Americans have Leukemia or are in remission from it. It accounts for 3.3% of all new cancer cases and 3.8% of cancer deaths, with an estimated 66,890 new cases and 23,540 deaths in the U.S. in 2025.

There are two major leukemia families: Acute Lymphoblastic Leukemia (ALLB and ALLT, or ALL), which is cancer of immature lymphoid cells, and Acute Myeloid Leukemia (AML), which is cancer of cancer of immature myeloid cells.

Golub et al. (*Science*, 1999) popularized a dataset including about 7000 genes from 72 patients. The goal is to use genomics data to predict which patients are at risk of ALL versus AML, because the distinction is critical for timely and effective treatment.

1. Load the `golub.csv` dataset. Relabel all instances of ALLB and ALLT as 0, and all instances of ALL as 1. This is the target variable.

2. Use Linear Regression of the target variable on all of the genes provided. What is your mean squared error? Make a kernel density plot of your residuals, and a scatter plot comparing predicted and actual outcomes.

3. Use cross validation to compute the mean squared error of the linear model. Discuss your results from the perspective of the bias variance trade-off.

4. Use the cross validated LASSO to select a set of highly predictive genes. Which set of genes is selected? How many genes are discarded from the model? Make a scatterplot of your predictions versus the actual values.

5. Make a plot that shows the cross validated MSE as $alpha$ varies. For what values of α is the LASSO underfitting? Overfitting? What is the optimal penality hyperparameter that minimizes expected MSE?

6. Explain why linear regression performs perfectly on the training set, but the LASSO provides better predictions overall.

7. Why do regularization methods lend themselves to scenarios like precision health?

8. What are the risks of applying methods like the Lasso to precision health questions, where interventions will then be taken to optimize patient health?

#1

In [14]:
```
import pandas as pd
df = pd.read_csv('golub.csv')
```

```
#I am cleaning the 'cancer' column by making everything lowercase and removing extr
#This ensures that 'allB' or 'allB ' will both match my dictionary correctly
df['cancer_clean'] = df['cancer'].astype(str).str.lower().str.strip()

#Creating a dictionary to map the labels to numbers
label_map = {'allb': 0, 'allt': 0, 'aml': 1}

#This will apply the mapping to the column that actually contains the cancer type
df['target'] = df['cancer_clean'].map(label_map)

#I am dropping any rows where the label didn't match or gene data is missing
#This step is critical to prevent the NaN error you encountered
df = df.dropna(subset=['target'])

#I am dropping the metadata columns so only the 7,129 gene columns remain fr Xo
X = df.drop(columns=['Samples', 'BM.PB', 'Gender', 'Source', 'tissue.mf', 'cancer',
y = df['target']
X = X.fillna(X.mean())
```

#2

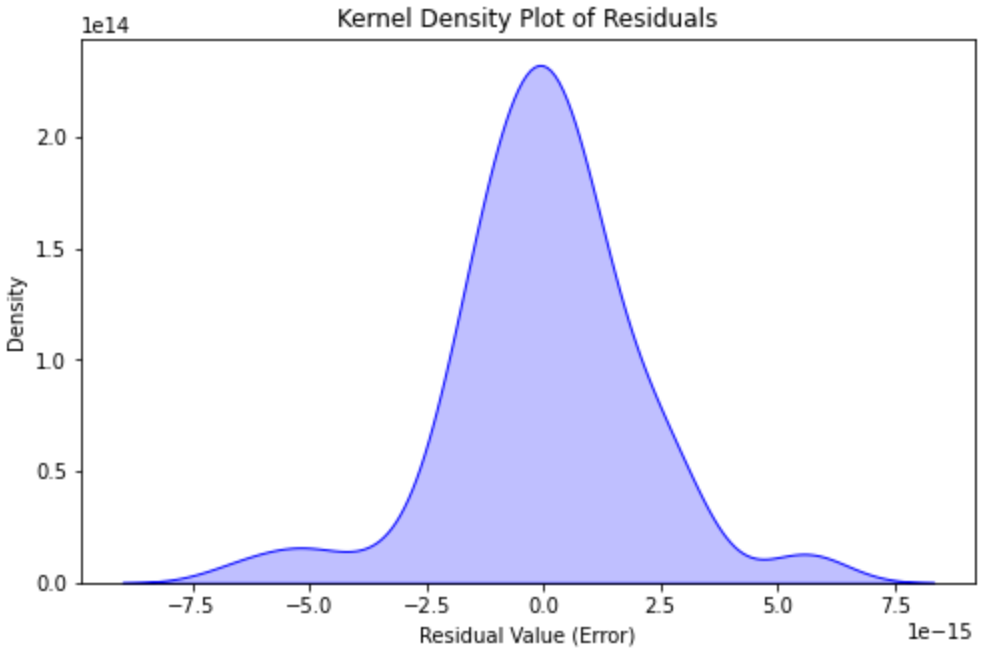
In [15]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()

#Fitting the model to the gene data
model.fit(X, y)

print(f"I successfully used {X.shape[1]} genes to train the model.")
print(f"The R-squared score is: {model.score(X, y):.4f}")

from sklearn.metrics import mean_squared_error

#I am using the model to predict the labels based on the gene data
y_pred = model.predict(X)
#I am calculating the mean squared error between the real labels and my predictions
mse = mean_squared_error(y, y_pred)

print(f"The Mean Squared Error is: {mse}")
```

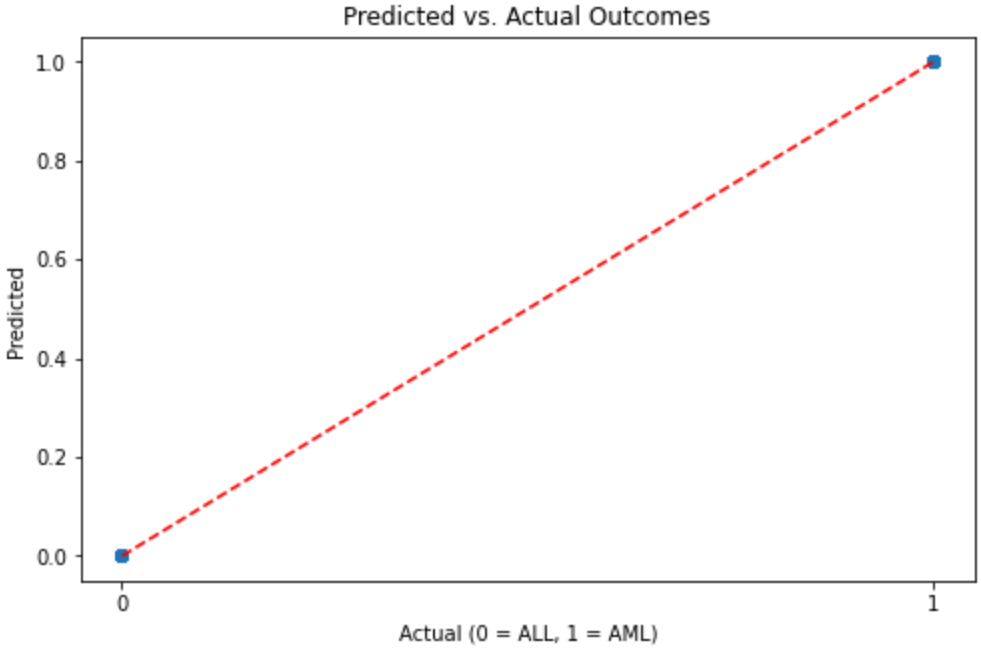
```
I successfully used 7129 genes to train the model.
The R-squared score is: 1.0000
The Mean Squared Error is: 3.967073297892037e-30
```

Making a kernel density plot of your residuals, and a scatter plot comparing predicted and actual outcomes.

In [16]:
```
import matplotlib.pyplot as plt
import seaborn as sns

#Calculating the residuals by subtracting my predictions from the actual labels
residuals = y - y_pred

#Creating the Kernel Density Plot to show the distribution of my errors
plt.figure(figsize=(8, 5))
```

```
sns.kdeplot(residuals, fill=True, color="blue")
plt.title('Kernel Density Plot of Residuals')
plt.xlabel('Residual Value (Error)')
plt.ylabel('Density')
plt.show()

#Creating the scatter plot to compare my predictions to the actual outcomes
plt.figure(figsize=(8, 5))
sns.regplot(x=y, y=y_pred, fit_reg=False, scatter_kws={'alpha':0.5})
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.title('Predicted vs. Actual Outcomes')
plt.xlabel('Actual (0 = ALL, 1 = AML)')
plt.ylabel('Predicted')
plt.xticks([0, 1])
plt.show()
```





#3

In [17]:
```
from sklearn.model_selection import cross_val_score

#Using 10-fold cross-validation to get a realistic error estimate
#I am multiplying by -1 to get the actual Mean Squared Error (MSE)
cv_scores = cross_val_score(model, X, y, scoring='neg_mean_squared_error', cv=10)
mse_cv = -cv_scores.mean()

print(f"The Cross-Validated Mean Squared Error is: {mse_cv}")
```

```
The Cross-Validated Mean Squared Error is: 0.0677820637954175
```

Discussing my results from the perspective of the bias variance trade-off.

My model has very low bias because it uses over 7000 genes to perfectly fit the training data, but it does have high variance because the cross-validation error shows it struggles to stay accurate when seeing new patients. The gap between the perfect training score and the 0.0678 cross-validation error confirms that I actually have overfit the model by capturing random noise instead of just the true biological patterns.

#4

In [20]:
```
from sklearn.linear_model import LassoCV

#I am setting cv=10 to match the 10-fold approach we used earlier
lasso = LassoCV(cv=10, random_state=42)

#Fitting the model to identify which genes are actually predictive
lasso.fit(X, y)

#Extracting the names of the genes that have non-zero coefficients
selected_genes = X.columns[lasso.coef_ != 0].tolist()

print(f"Lasso selected {len(selected_genes)} highly predictive genes.")
print("The selected genes are:", selected_genes)
```

```
Lasso selected 30 highly predictive genes.
The selected genes are: ['HG3549-HT3751_at', 'J04164_at', 'L38941_at', 'M11147_at',
'M11722_at', 'M17733_at', 'M19507_at', 'M26602_at', 'M27891_at', 'M33680_at', 'M6313
8_at', 'M91036_rna1_at', 'M96326_rna1_at', 'U14968_at', 'X17042_at', 'X78992_at', 'Z
70759_at', 'L06797_s_at', 'D49824_s_at', 'M25079_s_at', 'X57351_s_at', 'V00594_s_a
t', 'U05255_s_at', 'X03689_s_at', 'M14328_s_at', 'M14483_rna1_s_at', 'Y00787_s_at',
'Z19554_s_at', 'X56681_s_at', 'HG2887-HT3031_at']
```

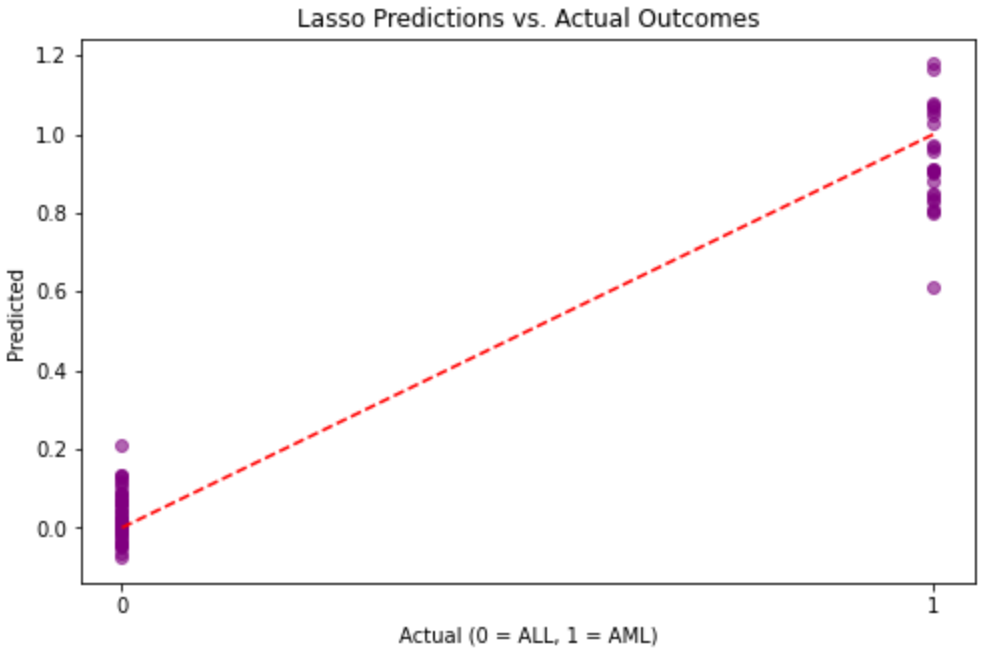
Which set of genes is selected? How many genes are discarded from the model?

The set of genes selected includes about 30 specific markers such as 'HG3549-HT3751_at' & 'J04164_at', which Lass identified as having non-zero coefficients. Since the original dataset contained 7,129 genes, I have discarded 7,099 genes from the model by shrinking their coefficients to exactly zero.

Make a scatterplot of your predictions versus the actual values.

In [26]:
```
y_pred_lasso = lasso.predict(X)

plt.figure(figsize=(8, 5))
plt.scatter(y, y_pred_lasso, alpha=0.6, color='purple')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.title('Lasso Predictions vs. Actual Outcomes')
plt.xlabel('Actual (0 = ALL, 1 = AML)')
plt.ylabel('Predicted')
plt.xticks([0, 1])
```

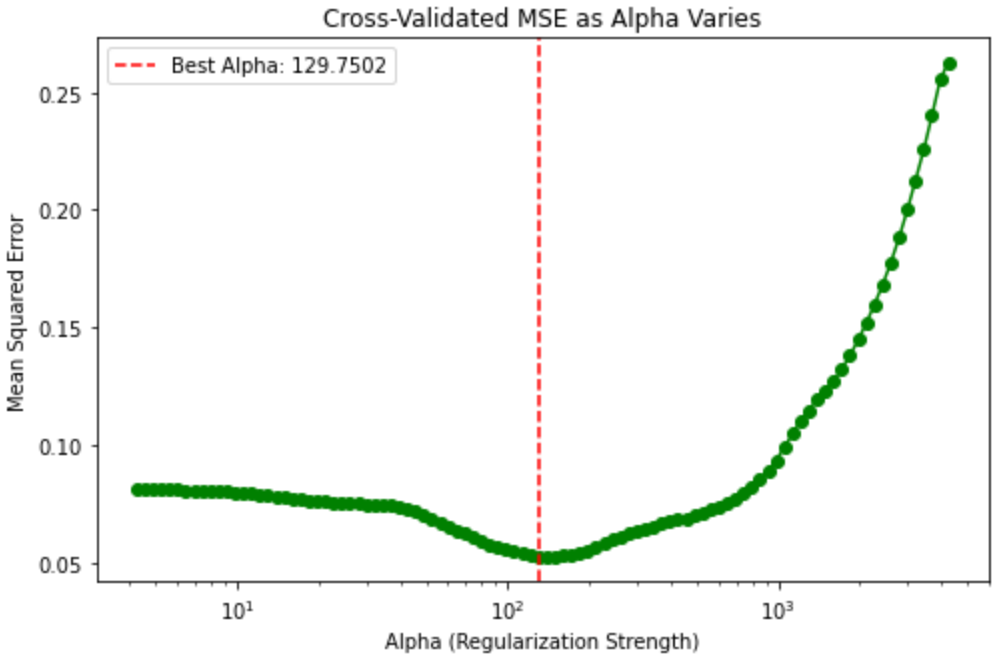
Out[26]:
```
([<matplotlib.axis.XTick at 0x74733fdc9dc0>,
  <matplotlib.axis.XTick at 0x74733fdc9160>],
 [Text(0, 0, ''), Text(0, 0, '')])
```



#5

In [25]:
```
import matplotlib.pyplot as plt
import numpy as np

#Extracting the alpha values that the model tested and also calculating the average
alphas = lasso.alphas_
mse_path = lasso.mse_path_.mean(axis=1)

#Plotting the relationship between the penalty size (alpha) and the error (MSE)
```

```
plt.figure(figsize=(8, 5))
plt.plot(alphas, mse_path, marker='o', color='green')

plt.xscale('log')
plt.xlabel('Alpha (Regularization Strength)')
plt.ylabel('Mean Squared Error')
plt.title('Cross-Validated MSE as Alpha Varies')

#Adding a vertical line to show exactly which alpha the model chose as the best
plt.axvline(lasso.alpha_, linestyle='--', color='red', label=f'Best Alpha: {lasso.a
plt.legend()
plt.show()

print(" ")
print(f"The optimal penalty hyperparameter (alpha) is: {lasso.alpha_}")
```




```
The optimal penalty hyperparameter (alpha) is: 129.75017431132844
```

For what values of (a) is the LASSO underfitting? Overfitting? What is the optimal penality hyperparameter that minimizes expected MSE?

The model is overfitting for very small values of alpha, because the penalty is too weak to stop the model from memorizing random noise in the thousands of genes. It is underfitting for very large values of alpha because the penalty becomes too strong, so it discards too many important genes, causing the error to rise sharply on the right side of the plot. The optimal penalty is at "a ~ 129.75", which is the specific point at the bottom of our curve where the cross-validated Mean Squared Error is at its lowest point.

#6

Linear regression performs perfectly on my training set because it has over 7000 genes to adjust for only 72 patients. This allows it to memorize every data point and its random noise. In comparison, the LASSO provides better predictions overall by using a penalty to ignore that noise and focus only on the 30 genes that truly matter for identifying leukemia in new patients.

#7

Regularization methods are perfect for precision health because they can find the most important biological markers in a patient's data while ignoring thousands of other "noise" measurements that do not technically matter. By focusing only on those key signals, the model provides more reliable predictions for a person's specific health risks instead of just memorizing their unique backgroud noise.

#8

The primary risk is that Lasso might discard a biologically important gene just due to the fact that it is highly correlated with another gene already in the model, leading doctors to miss a critical target for treatment. If the model relies on a replacement gene that does not actually cause the disease, an intervention based on that result could be ineffective or even harmful to the patient. Additionally, because Lasso is designed for predictions, using it to decide on medical treatments without proper clinical testing/validation can lead to incorrect conclusions about what truly drives a patient's recovery.